199. Binary Tree Right Side View

Solved

Medium | Topics | Companies

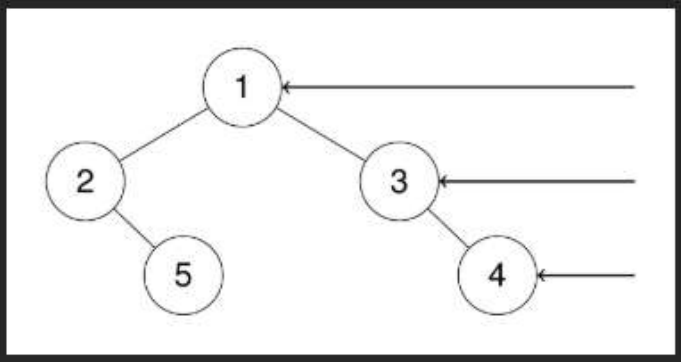
Given the `root` of a binary tree, imagine yourself standing on the **right side** of it, return *the values of the nodes you can see ordered from top to bottom.*

Example 1:

Input: `root = [1,2,3,null,5,null,4]`

Output: `[1,3,4]`

Explanation:



Example 2:

Input: `root = [1,2,3,4,null,null,null,5]`

Output: `[1,3,4,5]`

Code | C++

```cpp
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left)
 * };
 */

class Solution {
public:
    vector<int> rightSideView(TreeNode* root) {
        vector<int> ans;
        if (root == nullptr)
            return ans;
        queue<TreeNode*> q;
        q.push(root);
        while (!q.empty()) {
            int size = q.size();
            for (int i = 0; i < size; i++) {
                TreeNode* node = q.front();
                q.pop();
                if (node->left != nullptr)
                    q.push(node->left);
                if (node->right != nullptr)
                    q.push(node->right);
                if (i == size - 1)
                    ans.push_back(node->val);
            }
        }
        return ans;
    }
};

```

View less

2697. Lexicographically Smallest Palindrome

Solved

Easy | Topics | Companies | Hint

You are given a string `s` consisting of **lowercase English letters**, and you are allowed to perform operations on it. In one operation, you can **replace** a character in `s` with another lowercase English letter.

Your task is to make `s` a **palindrome** with the **minimum number of operations** possible. If there are **multiple palindromes** that can be made using the **minimum** number of operations, make the **lexicographically smallest** one.

A string `a` is lexicographically smaller than a string `b` (of the same length) if in the first position where `a` and `b` differ, string `a` has a letter that appears earlier in the alphabet than the corresponding letter in `b`.

Return *the resulting palindrome string.*

Example 1:

> **Input:** s = "egcfe"
> **Output:** "efcfe"
> **Explanation:** The minimum number of operations to make "egcfe" a palindrome is 1, and the lexicographically smallest palindrome string we can get by modifying one character is "efcfe", by changing 'g'.

C++ | Auto

```cpp
class Solution {
public:
    string makeSmallestPalindrome(string s) {
        int n = s.length();
        for (int i = 0; i < n / 2; i++) {
            int j = n - 1 - i;
            if (s[i] < s[j]) {
                s[j] = s[i];
            }
            else if (s[i] > s[j]) {
                s[i] = s[j];
            }
        }
        return s;
    }
};
```

114. Flatten Binary Tree to Linked List

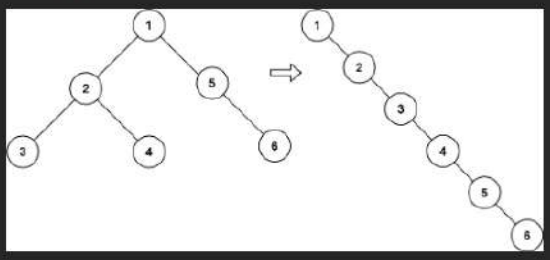
Solved

Medium | Topics | Companies | Hint

Given the `root` of a binary tree, flatten the tree into a "linked list":

- The "linked list" should use the same `TreeNode` class where the `right` child pointer points to the next node in the list and the `left` child pointer is always `null`.
- The "linked list" should be in the same order as a pre-order traversal of the binary tree.

Example 1:



Input: root = [1,2,5,3,4,null,6]
Output: [1,null,2,null,3,null,4,null,5,null,6]

```
*     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x)
* };
*/

class Solution {
public:
    void flatten(TreeNode* root) {
        TreeNode* curr = root;
        while (curr != nullptr) {
            if (curr->left != nullptr) {
                TreeNode* prev = curr->left;
                while (prev->right != nullptr) {
                    prev = prev->right;
                }
                prev->right = curr->right;
                curr->right = curr->left;
                curr->left = nullptr;
            }
            curr = curr->right;
        }
    }
};

```

Saved

236. Lowest Common Ancestor of a Binary Tree

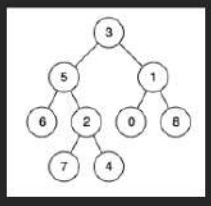
Solved

Medium | Topics | Companies

Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree.

According to the definition of LCA on Wikipedia: "The lowest common ancestor is defined between two nodes `p` and `q` as the lowest node in `T` that has both `p` and `q` as descendants (where we allow **a node to be a descendant of itself**)."

Example 1:



Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1
Output: 3
Explanation: The LCA of nodes 5 and 1 is 3.

C++ | Auto

```
 *     TreeNode *right;
 *     TreeNode(int x) : val(x), left(NULL), right(NULL) {}
 * };
 */

class Solution {
public:
    TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
        if (root == nullptr || root == p || root == q) {
            return root;
        }
        TreeNode* left = lowestCommonAncestor(root->left, p, q);
        TreeNode* right = lowestCommonAncestor(root->right, p, q);
        if (left != nullptr && right != nullptr) {
            return root;
        }
        if (left != nullptr) {
            return left;
        }
        return right;
    }
};
```

129. Sum Root to Leaf Numbers

Solved

Medium | Topics | Companies

You are given the `root` of a binary tree containing digits from `0` to `9` only.

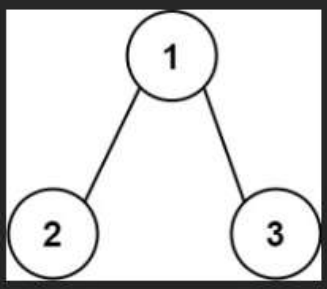
Each root-to-leaf path in the tree represents a number.

- For example, the root-to-leaf path `1 -> 2 -> 3` represents the number `123`.

Return *the total sum of all root-to-leaf numbers*. Test cases are generated so that the answer will fit in a **32-bit** integer.

A **leaf** node is a node with no children.

Example 1:




```
Input: root = [1,2,3]
Output: 25
Explanation:
The root-to-leaf path 1->2 represents the number 12.
The root-to-leaf path 1->3 represents the number 13.
Therefore, sum = 12 + 13 = 25.
```

C++ | Auto

```cpp
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x),
 * };
 */

class Solution {
public:
    int sumNumbers(TreeNode* root) {
        return dfs(root, 0);
    }
    int dfs(TreeNode* node, int currentNumber) {
        if (node == nullptr) {
            return 0;
        }
        currentNumber = currentNumber * 10 + node->val;
        if (node->left == nullptr && node->right == nullptr) {
            return currentNumber;
        }
        return dfs(node->left, currentNumber)
             + dfs(node->right, currentNumber);
    }
};
```

345. Reverse Vowels of a String

Solved

Easy | Topics | Companies

Given a string `s`, reverse only all the vowels in the string and return it.

The vowels are `'a'`, `'e'`, `'i'`, `'o'`, and `'u'`, and they can appear in both lower and upper cases, more than once.

Example 1:

> **Input:** `s = "IceCreAm"`
>
> **Output:** `"AceCreIm"`
>
> **Explanation:**
>
> The vowels in `s` are `['I', 'e', 'e', 'A']`. On reversing the vowels, s becomes `"AceCreIm"`.

Example 2:

> **Input:** `s = "leetcode"`
>
> **Output:** `"leotcede"`

Constraints:

- `1 <= s.length <= 3 * 10^5`
- `s` consist of **printable ASCII** characters.

C++ | Auto

```cpp
class Solution {
public:
    bool isVowel(char c) {
        return c == 'a' || c == 'e' ||
               c == 'i' || c == 'o' ||
               c == 'u' || c == 'A' ||
               c == 'E' || c == 'I' ||
               c == 'O' || c == 'U';
    }
    string reverseVowels(string s) {
        int left = 0;
        int right = s.length() - 1;
        while (left < right) {
            while (left < right && !isVowel(s[left])) {
                left++;
            }
            while (left < right && !isVowel(s[right])) {
                right--;
            }
            if (left < right) {
                swap(s[left], s[right]);
                left++;
                right--;
            }
        }
        return s;
    }
};
```

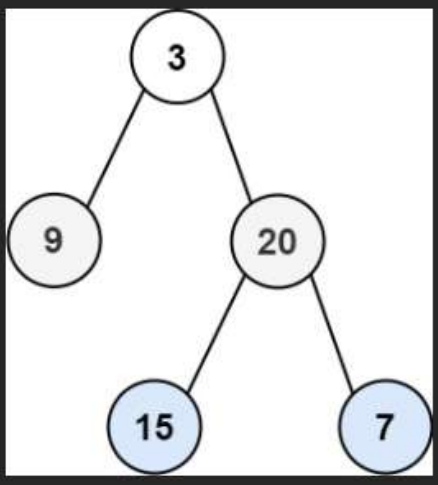
103. Binary Tree Zigzag Level Order Traversal

Solved

Medium | Topics | Companies

Given the `root` of a binary tree, return *the zigzag level order traversal of its nodes' values.* (i.e., from left to right, then right to left for the next level and alternate between).

Example 1:




```
Input: root = [3,9,20,null,null,15,7]
Output: [[3],[20,9],[15,7]]
```

Example 2:

```
Input: root = [1]
Output: [[1]]
```

```cpp
 * };
 */

class Solution {
public:
    vector<vector<int>> zigzagLevelOrder(TreeNode* root) {
        vector<vector<int>> result;
        if (root == NULL) {
            return result;
        }
        queue<TreeNode*> q;
        q.push(root);
        bool leftToRight = true;
        while (!q.empty()) {
            int size = q.size();
            vector<int> level(size);
            for (int i = 0; i < size; i++) {
                TreeNode* node = q.front();
                q.pop();
                int index = leftToRight ? i : size - 1 - i;
                level[index] = node->val;
                if (node->left != NULL) {
                    q.push(node->left);
                }
                if (node->right != NULL) {
                    q.push(node->right);
                }
            }
            result.push_back(level);
            leftToRight = !leftToRight;
        }
        return result;
    }
};
```

2108. Find First Palindromic String in the Array

Solved

Easy | Topics | Companies | Hint

Given an array of strings `words`, return *the first **palindromic** string in the array*. If there is no such string, return *an **empty string** `""`*.

A string is **palindromic** if it reads the same forward and backward.

Example 1:

> **Input:** words = ["abc","car","ada","racecar","cool"]
> **Output:** "ada"
> **Explanation:** The first string that is palindromic is "ada".
> Note that "racecar" is also palindromic, but it is not the first.

Example 2:

> **Input:** words = ["notapalindrome","racecar"]
> **Output:** "racecar"
> **Explanation:** The first and only string that is palindromic is "racecar".

Example 3:

> **Input:** words = ["def","ghi"]
> **Output:** ""
> **Explanation:** There are no palindromic strings, so the empty string is returned.

```cpp
class Solution {
public:
    bool isPalindrome(string& word) {
        int left = 0;
        int right = word.length() - 1;
        while (left < right) {
            if (word[left] != word[right]) {
                return false;
            }
            left++;
            right--;
        }
        return true;
    }
    string firstPalindrome(vector<string>& words) {
        for (string& word : words) {
            if (isPalindrome(word)) {
                return word;
            }
        }
        return "";
    }
};
```